

MetaRoCE in Software



Ashkan Nikravesh *Software Engineer, Meta Platforms, Inc.*

Yiorgos Nikolaidis *Software Engineer, Meta Platforms, Inc.*

What MetaRoCE is

The Problem

Problems with RoCEv2 Deployments

PFC pauses a whole traffic class, so one slow receiver stops every sender in it. And RoCEv2 puts a connection on a single flow, so ECMP has nothing to spread across: one link fills up while the parallel links sit idle.

Best-Effort Ethernet

A hardware-offloadable RDMA transport for best-effort Ethernet. It only asks for ECN marking on a WRED-like ramp rather than losslessness.

Multipath Inside QP

Multipath handling inside a single Queue Pair (QP), meaning path diversity stops being the application's problem.

Out-of-Order Tracking

Selective acknowledgment allows precise tracking. A single lost packet costs exactly one packet rather than an entire window.

Stream Multiplexing

Per-tag stream multiplexing ensures that one blocked stream does not stall the other streams sharing the same connection.

What MetaRoCE is

The Problem

Problems with RoCEv2 Deployments

PFC pauses a whole traffic class, so one slow receiver stops every sender in it. And RoCEv2 puts a connection on a single flow, so ECMP has nothing to spread across: one link fills up while the parallel links sit idle.

In-band failure mitigation

Network failures are handled in-band, with the multi-flow logic avoiding faulty links

End-host rebalancing

The multi-flow logic also rebalanes load across links without the end-hosts needing information from a centralized controller.

Incast mitigation

The Rate Control Notification mechanism informs senders about last hop congestion and adjusts their sending rate.

Long distance

MetaRoCE is designed to work across remote sites allowing us to build multi-site clusters.

More on MetaRoCE

Two Networking @SCALE talks covering MetaRoCE in depth:





You can't test against a PDF

The Spec Prose

A transport spec is a few hundred pages of prose about behaviour under conditions the prose cannot produce.

Pre-Silicon

Hardware teams need something to bring up against before silicon exists to accelerate development.

Researchers

Researchers need somewhere to try an idea in days rather than waiting for slow tapeouts.

The Tiebreak

Everyone needs an ultimate tiebreak when two independent boxes disagree about who is wrong.

So we implemented it in software (twice)

libsoftmetaroce

RUNS ANYWHERE

Pure user space over ordinary sockets on any Linux box. Observable by construction: log transitions, single-step handshakes, and inject exact packet degradations.

dpdkmetaroce

BUILT FOR PERFORMANCE

Kernel bypass, poll-mode drivers, and pluggable underlay. Layered deliberately so that the software seams fall precisely where hardware boundaries exist.

Two Speed Classes

Kernel bypass makes dpdkmetaroce fundamentally faster. However, libsoftmetaroce still reaches the heavy rates where buffers fill and congestion occurs.

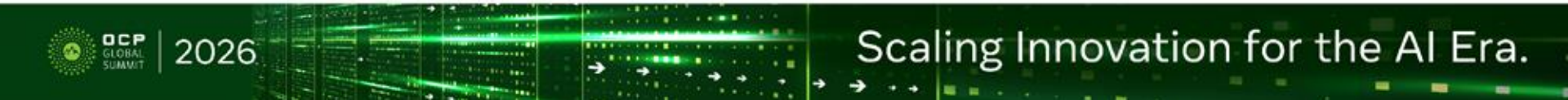
Writing Changes Spec

Implementing features forces hard questions prose never answers. Several logic fixes and clarifications went straight back into the specification.

The Ultimate Tiebreak

By pointing two independent implementations of the same protocol at each other, any disagreement immediately highlights a bug or a spec ambiguity.

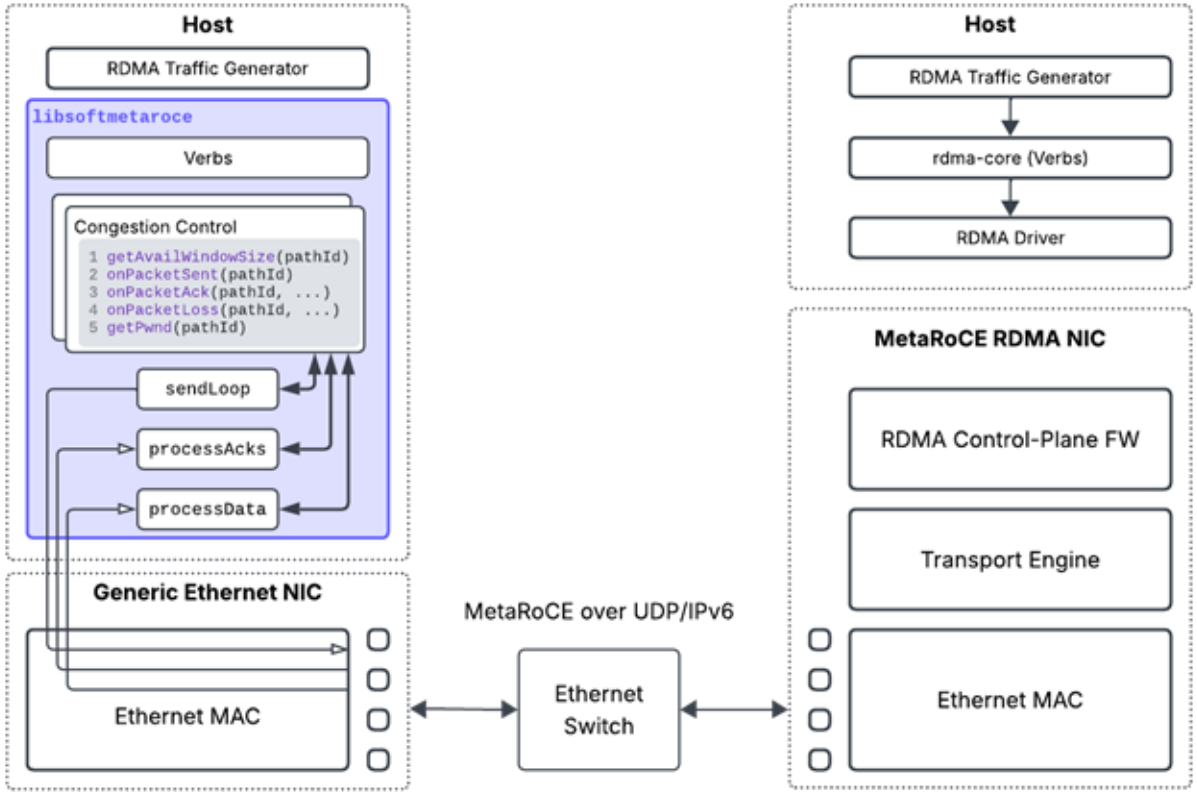
And we are open-sourcing it



libsoftmetaroce

Complete Transport

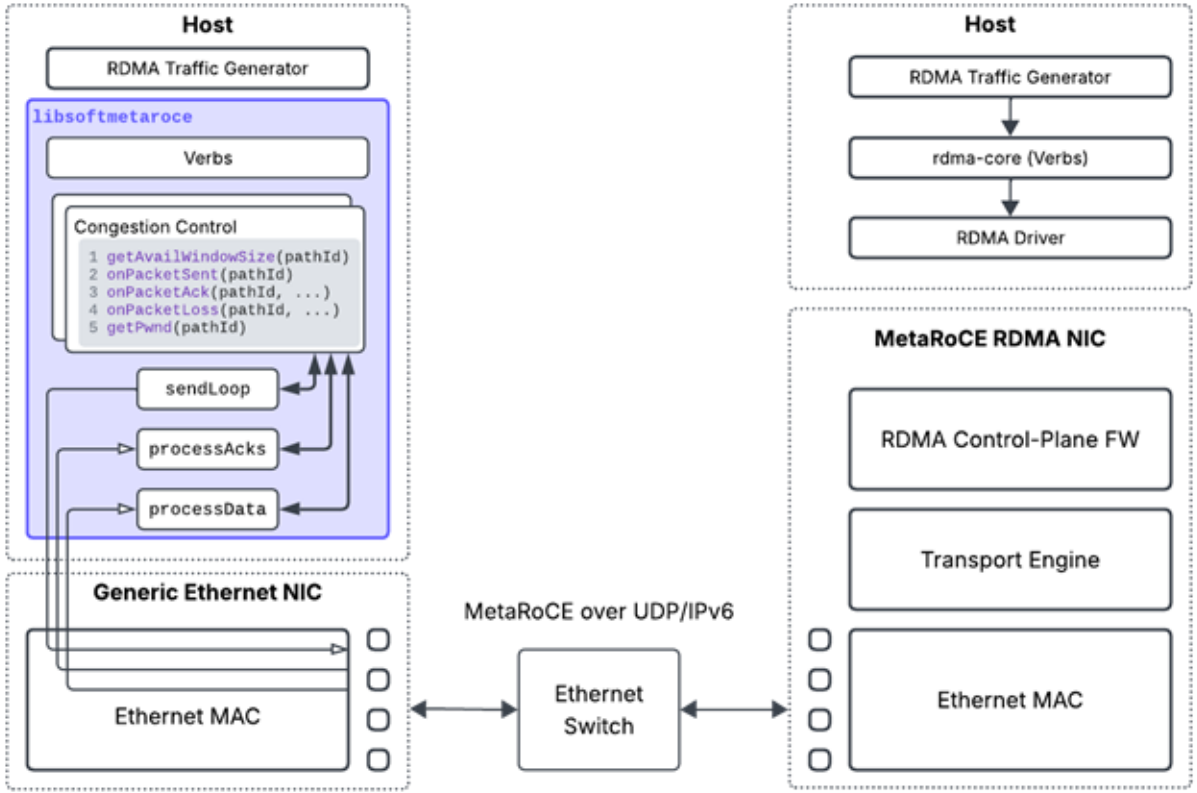
A complete software implementation of the transport, not a packet parser: connection setup, reliable delivery, multipath, per-path congestion control, retransmission.



libsoftmetaroce

Pure User Space

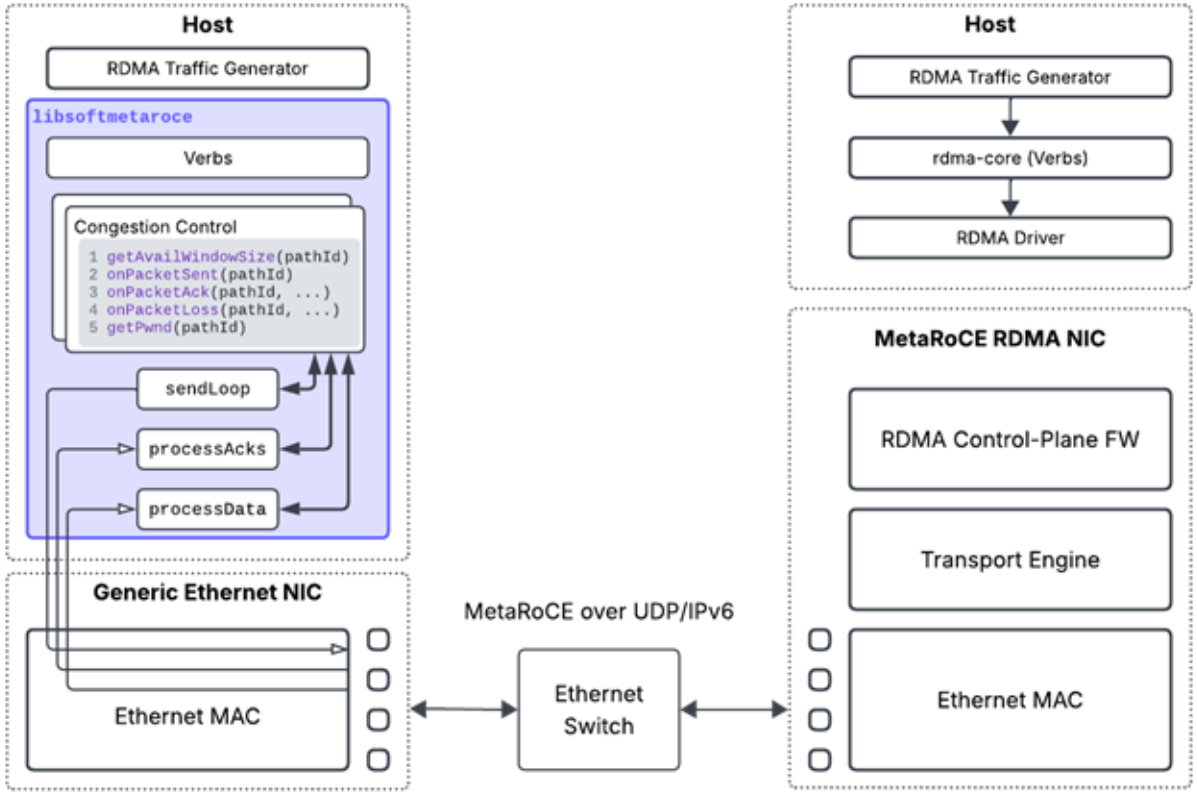
Pure user space over ordinary UDP sockets. No RDMA NIC, no special drivers. A laptop or a CI container is enough to run the entire stack.



libsoftmetaroce

Standard Verbs

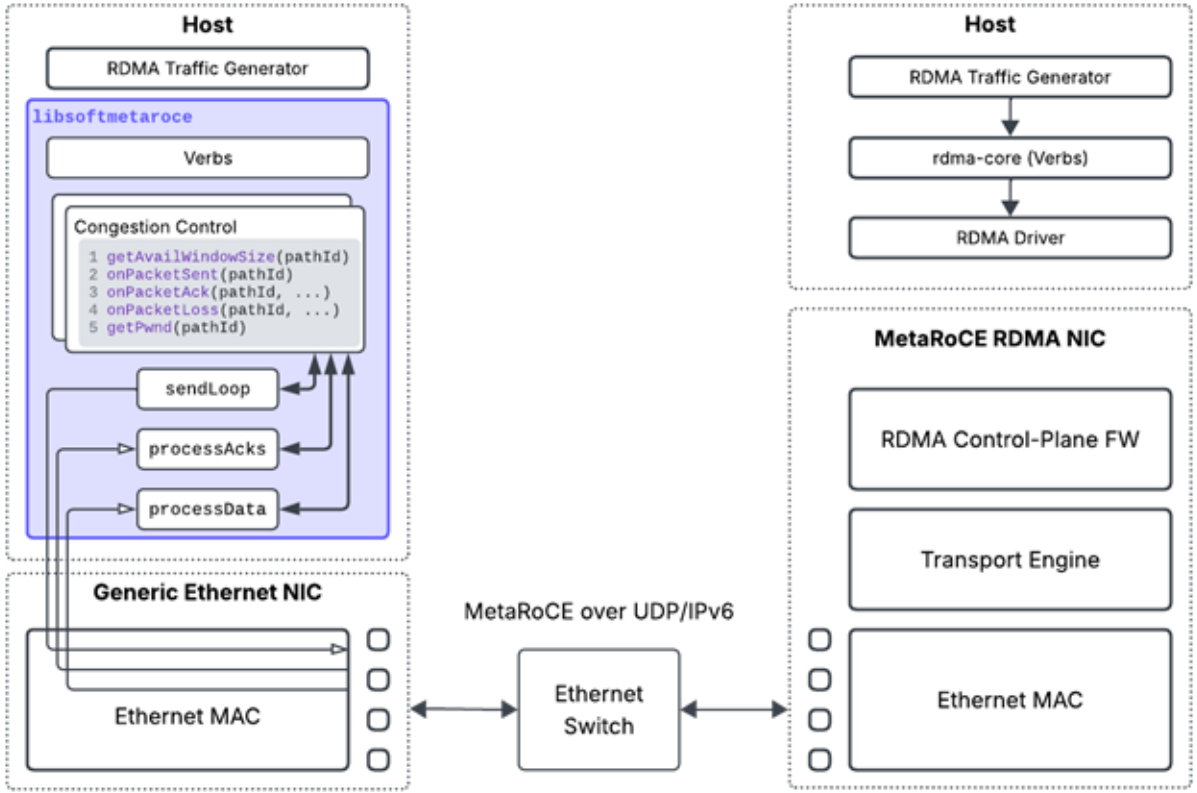
Standard verbs API, allowing existing RDMA applications and benchmarks to compile and run against the software transport entirely unchanged.



libsoftmetaroce

Observable

Observable by construction: log every state transition, single-step through handshakes, and inject deliberate reorders at an exact packet.



libsoftmetaroce: the part you can swap

Multipath

A path is a UDP socket, a port is a local IP address. Many paths across several planes on one ordinary host, with no special hardware.

Congestion control

Three events define the interface: packet sent, packet acked, packet lost. A new algorithm is one C++ class.

Wire envelope

One seam adds framing on transmit and validates it on receive. UDP and direct L2 Ethernet today; built to take OCP ESUN and PSP.

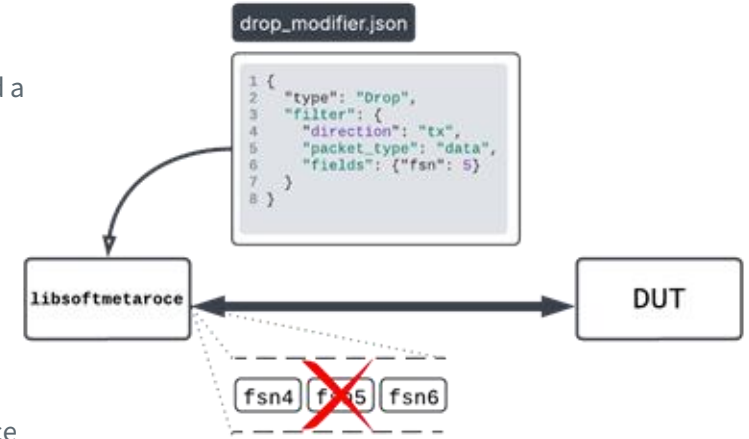
libsoftmetaroce: break it on purpose

Modifiers

With modifiers, any packet can be dropped, delayed, duplicated, or reordered by rule.

Config-Driven

Congestion, loss, and reordering scenarios that would need a lab and a traffic generator, expressed as configuration.



Deterministic

A retransmission test that fails the same way every run, not a flakey implementation you re-run until it passes.

Read for This

When bringing hardware up before silicon exists, running a conformance suite, testing a congestion control idea, or reproducing a rare wire condition on demand.

dpdkmetaroce

Independently written

Shares no code with libsoftmetaroce. That independence is what makes the pair a cross-check.

Kernel bypass & PMDs

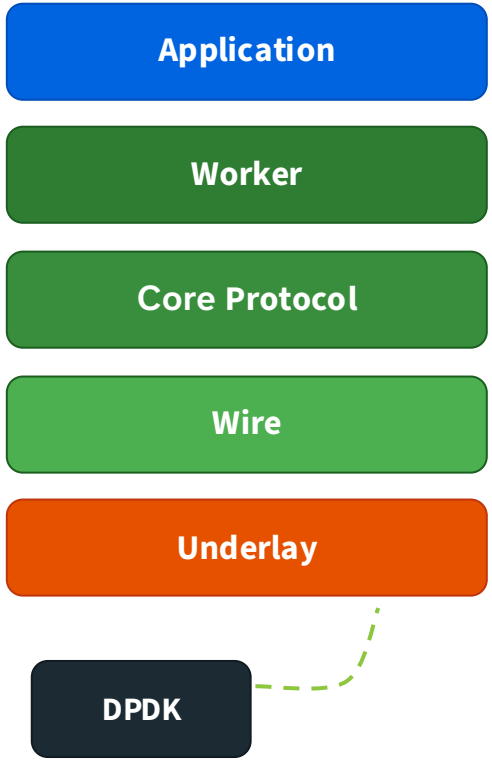
Runs the protocol at rate using poll-mode drivers, making it fundamentally faster than libsoftmetaroce.

Layered at hardware boundaries

Seams fall precisely where hardware boundaries do: wire codec, transport state machine, and connection management.

Standard verbs API

Maintains API compatibility so that identical applications and benchmarks can run seamlessly against either implementation.



dpdkmetaroce: any datapath

Narrow Interface

One narrow interface sits below the transport. Behind it today: DPDK poll-mode drivers, a vendor DPU datapath, and ordinary sockets for tests.

Simple Integration

Supporting new hardware means implementing that interface, not forking a transport. We have run the same transport over more than one class of device this way.

Immediate Adoption

It puts MetaRoCE on hardware that does not implement it. A device with a supported driver speaks the protocol today, without waiting for silicon.

Reach For This

Reach for this one when you need the protocol driven at rate, or when you want MetaRoCE on a device that has no hardware support for it.

How we know it is right

Continuous Integration

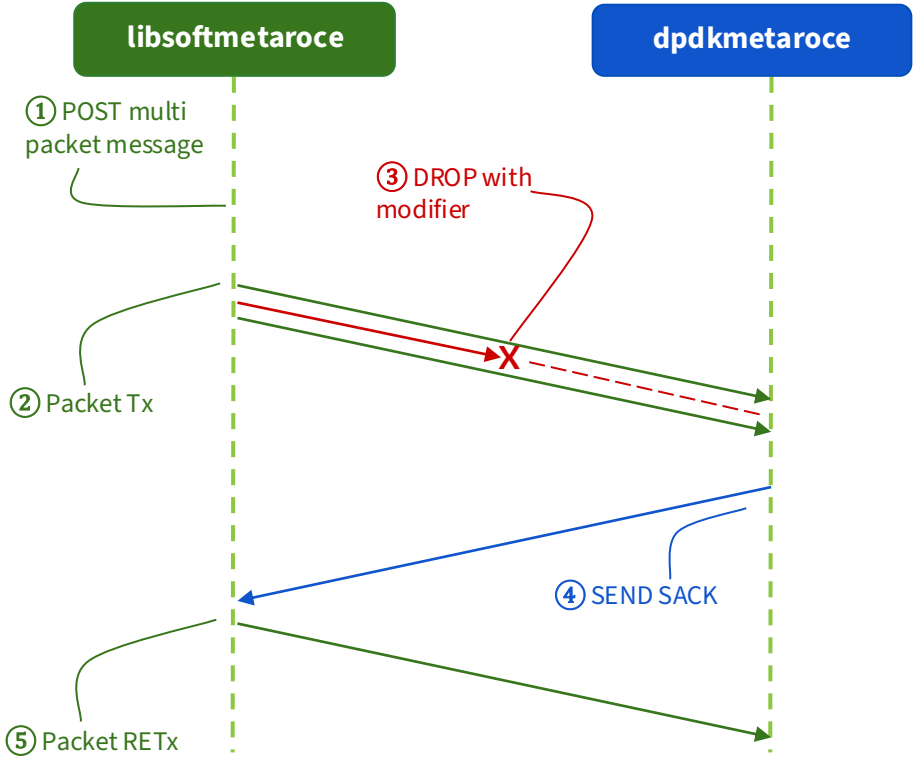
Both implementations run their suites on every change, including awkward corners like atomics, error escalation, and out-of-order delivery.

Cross-Implementation Matrix

Every pairing is exercised (socket-to-socket, DPDK-to-DPDK, and cross-pairs) to actively catch spec drift.

Fault Injection

The modifier layer ensures recovery paths are thoroughly tested rather than just assumed.



What we are open-sourcing

Both implementations

Under a permissive licence, with a standard build.

Standard Verbs API

Existing RDMA applications and benchmarks run against them completely unchanged.

No special requirements

The implementations need nothing but a Linux box: two endpoints on one machine. You can use dpdkmetaroce without RDMA NIC and no special drivers if you want.

Getting started

Checkout MetaRoCE GitHub.

https://github.com/facebookincubator/metaroce

Scan to open

Call to action

Run it

Two endpoints on one Linux box is the whole setup. Nothing here needs an RDMA NIC to try.

Point your implementation to ours

If the two disagree, either one of us has a bug or the specification is ambiguous.

Let us know

We want community input to make MetaRoCE better.

Thank You!

